

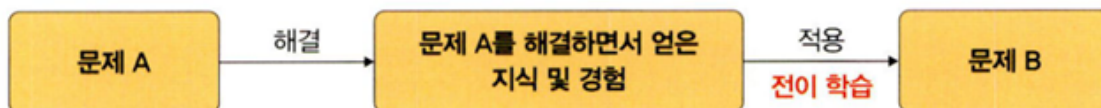
<https://jade0728.tistory.com/63> 에서 확인 가능합니다.

5.3 전이학습

CNN을 제대로 훈련시키려면 많은 양의 데이터가 필요 → 전이학습 **transfer learning**

transfer learning: 이미지넷(ImageNet) 처럼 큰 데이터셋을 써서 훈련된 모델의
가중치를 가져와 우리가 해결하려는 과제에 맞게 보정해서 사용하는 것

▼ 그림 5-29 전이 학습

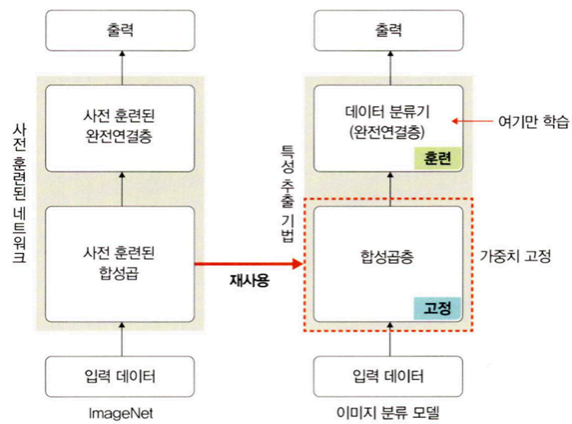


5.3.1 특성 추출 기법 feature extractor

ImageNet 데이터셋으로 사전 훈련된 모델을 가져온 후 마지막에 완전연결층 부분만 새로 만든다. 사전훈련된 네트워크의 합성곱층에 새로운 데이터를 통과시키고 그 출력을 데이터 분류기에서 훈련시킨다.

사용가능한 이미지 분류 모델은 Xception, Inception V3, ResNet50, VGG16, VGG19, MobileNet 등이 있다.

♥ 그림 5-30 특성 추출 기법



코드를 통해 알아보자

1. 데이터를 전처리

```
data_path='../chap5/data/catanddog/train'

transform=transforms.Compose(
    [
        transforms.Resize([256,256]),
        transforms.RandomResizedCrop(224),
        transforms.RandomHorizontalFlip(),
        transforms.ToTensor(),
    ]
)

train_dataset=torchvision.datasets.ImageFolder(
    data_path, transform=transform
)
train_dataloader=DataLoader(
    train_dataset, batch_size=32, num_workers=8, shuffle=True
)

print(len(train_dataset))
```

- **Resize:** 이미지 크기를 조정
- **RandomResizedCrop:** 이미지를 랜덤한 크기와 비율로 자른다.
- **RandomHorizontalFlip:** 이미지를 랜덤하게 수평으로 뒤집는다.
- **ToTensor:** 이미지 데이터를 텐서로 변환한다.
- **datasets.ImageFolder:** 데이터로더가 데이터를 불러올 대상과 방법을 정의

- **num_workers=8** : 데이터를 불러올 때 하위 프로세스를 몇 개 사용할지를 설정, 너무 많으면 메모리 부족 현상이 발생할 수도 있음
- **shuffle**: 데이터를 무작위로 섞어서 랜덤으로 불러옴

2. 학습에 사용될 이미지 출력

```

samples, labels= iter(train_loader).next()
classes={0:'cat', 1:'dog'}
fig=plt.figure(figsize=(16,24))
for i in range(24):
    a=fig.add_subplot(4,6, i+1)
    a.set_title(classes[labels[i].item()])
    a.axis('off')
    a.imshow(np.transpose(samples[i].numpy(), (1,2,0)))

plt.subplots_adjust(bottom=0.2, top=0.6, hspace=0)

```

- **np.transpose**: 행과 열을 바꿔서 행렬의 차원을 바꿔줌 for 행렬의 내적 연산, 행렬 계산을 위해 차원을 조정

3. 사전 훈련된 모델 내려받기

```
resnet18=models.resnet18(pretrained=True)
```

- **ResNet18**: 50개의 계층으로 이루어진 CNN, ImageNet 데이터베이스의 100만개가 넘는 영상을 이용하여 훈련된 신경망으로 전이 학습에 사용되도록 사전 훈련된 모델을 제공하고 있다.

▲ PyTorch의 torchvision에 이미 구현되어 있는 유명한 이미지 딥러닝 모델들을 불러와서 생성할수 있다.

```

import torchvision.models as models

resnet18 = models.resnet18()
alexnet = models.alexnet()
vgg16 = models.vgg16()
squeezenet = models.squeezenet1_0()
densenet = models.densenet161()

```

```

inception = models.inception_v3()
googlenet = models.googlenet()
shufflenet = models.shufflenet_v2_x1_0()
mobilenet_v2 = models.mobilenet_v2()
mobilenet_v3_large = models.mobilenet_v3_large()
mobilenet_v3_small = models.mobilenet_v3_small()
resnext50_32x4d = models.resnext50_32x4d()
wide_resnet50_2 = models.wide_resnet50_2()

mnasnet = models.mnasnet1_0()

```

▲ **pretrained=True**를 통해 사전 학습된 모델과 모델의 가중치를 사용할 수 있다.

```

import torchvision.models as models

resnet18 = models.resnet18(pretrained=True)
alexnet = models.alexnet(pretrained=True)
squeezenet = models.squeezenet1_0(pretrained=True)
vgg16 = models.vgg16(pretrained=True)
densenet = models.densenet161(pretrained=True)

inception = models.inception_v3(pretrained=True)
googlenet = models.googlenet(pretrained=True)
shufflenet = models.shufflenet_v2_x1_0(pretrained=True)
mobilenet_v2 = models.mobilenet_v2(pretrained=True)
mobilenet_v3_large = models.mobilenet_v3_large(pretrained=True)
mobilenet_v3_small = models.mobilenet_v3_small(pretrained=True)
resnext50_32x4d = models.resnext50_32x4d(pretrained=True)
wide_resnet50_2 = models.wide_resnet50_2(pretrained=True)

mnasnet = models.mnasnet1_0(pretrained=True)

```

4. 사전 훈련된 모델의 파라미터 학습 유무 지정

```

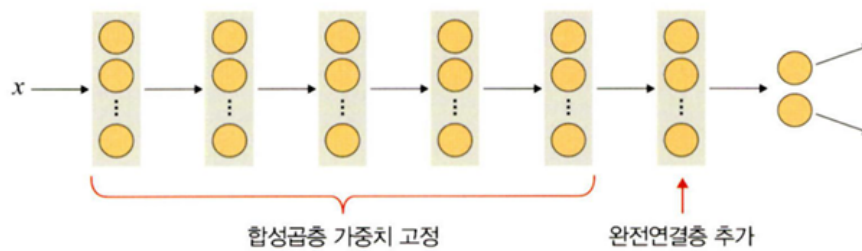
def set_parameter_requires_grad(model, feature_extracting=True):
    if feature_extracting:
        for param in model.parameters():
            param.requires_grad=False

set_parameter_requires_grad(resnet18)

```

- **requires_grad=False**: 모델의 일부(합성곱층, 풀링층)를 고정하고 나머지를 학습할 때

▼ 그림 5-36 ResNet18에 완전연결층 추가



5. ResNet18에 완전 연결층 추가, 모델의 파라미터 값 확인

```
resnet18.fc=nn.Linear(512,2)
for name, param in resnet18.named_parameters():
    if param.requires_grad:

        print(name, param.data)
```

6. 모델 객체 생성 및 손실 함수 정의

```
model=models.resnet18(pretrained=True)

#모델의 합성곱층 가중치 고정
for param in model.parameters():
    param.requires_grad=False

#완전 연결층 학습
model.fc=torch.nn.Linear(512,2)
for param in model.fc.parameters():
    param.requires_grad=True

optimizer=torch.optim.Adam(model.fc.parameters())
cost=torch.nn.CrossEntropyLoss() # 손실함수 정의

print(model)
```

7. 모델 학습을 위한 함수 생성

```
def train_model(modeel, dataloaders, criterion, optimizer, device,
num_epochs=13, is_train=True):
    since=time.time() # 컴퓨터의 현재 시각을 구하는 함수
    acc_history=[]
    loss_history=[]
    best_acc=0.0

    for epoch in range(num_epochs):
        print('Epoch{}/{}'.format(epoch, num_epochs-1))
        print('-'*10)
```

```

    running_loss=0.0
    running_corrects=0

    for inputs, labels in dataloaders:
        input=inputs.to(device)
        labels=labels.to(device)

        model.to(device)
        optimizer.zero_grad()
        outputs=model(inputs)
        loss=criterion(outputs, labels)
        _, preds=torch.max(outputs,1)
        loss.backward()
        optimizer.step()

        #출력결과와 레이블의 오차를 계산한 결과를 누적해서 저장
        running_loss+=loss.item()*inputs.size(0)
        #출력결과와 레이블이 동일한지 확인한 결과를 누적하여 저장
        running_corrects+=torch.sum(preds==labels.data)

    # 평균 오차, 정확도 계산
    epoch_loss=running_loss/len(dataloaders.dataset)
    epoch_acc=running_corrects.double()/len(dataloaders.dataset)

    print('Loss: {:.4f} Acc: {:.4f}'.format(epoch_loss, epoch_acc))

    if epoch_acc>best_acc:
        best_acc=epoch_acc

    acc_history.append(epoch_acc.item())
    loss_history.append(epoch_loss)
    #모델 재사용을 위해 저장
    torch.save(model.state_dict(),
os.path.join('../chap05/data/catanddog/',
'{0:0=2d}.pth'.format(epoch)))
    print()

    time_elapsed=time.time() -since
    print('Training complete in {:.0f}m {:.0f}s'.format(time_elapsed//60,
time_elapsed %60))
    print('Best Acc: {:.4f}'.format(best_acc))

    return acc_history, loss_history

```

8. 파라미터 학습 결과를 **optimizer**에 전달: **fc.weight, fc.bias**

```

params_to_update=[]
for name, param in resnet18.named_parameters():

```

```

if param.requires_grad==True:
    params_to_update.append(param)
    print("\t", name)

```

```
optimizer=optim.Adam(params_to_update)
```

9. 모델 학습

```

device=torch.device("cuda" if torch.cuda.is_available() else "cpu")
criterion=nn.CrossEntropyLoss()

```

```

train_acc_hist, train_loss_hist=train_model(resnet18, train_loader,
criterion, optimizer, device)

```

10. 테스트 데이터 호출 및 전처리

```
test_path="../chap05/data/catanddog/test"
```

```

transform=transforms.Compose(
    [
        transforms.Resize(224),
        transforms.CenterCrop(224),
        transforms.ToTensor(),
    ]
)

```

```

test_dataset=torchvision.datasets.ImageFolder(
    root=test_path,
    transform=transform
)
train_loader=torch.utils.data.DataLoader(
    test_dataset, batch_size=32, num_workers=1, shuffle=True
)

```

```
print(len(test_dataset))
```

11. 테스트 데이터 평가 함수 생성

```

def eval_model(model, dataloaders, device):
    since=time.time()
    acc_history=[]
    best_acc=0.0

    saved_models=glob.glob('../chap05/data/catanddog/'+ '*.pth')
    saved_models.sort()

```

```

print('saved_model', saved_models)

for model_path in saved_models:
    print('loading model', model)

    model.load_state_dict(torch.load(model_path))
    model.eval()
    model.to(device)
    running_corrects=0

    for inputs, labels in dataloaders:
        inputs=inputs.to(device)
        labels=labels.to(device)

        with torch.no_grad():
            outputs=model(inputs)

            _, preds=torch.max(outputs.data, 1)
            preds[preds>=0.5]=1
            preds[preds<0.5]=0
            running_corrects+=preds.eq(labels.cpu()).int().sum()

    epoch_acc=running_corrects.double()/len(dataloaders.dataset)
    print('Acc: {:.4f}'.format(epoch_acc))

    if epoch_acc>best_acc:
        best_acc=epoch_acc

    acc_history.append(epoch_acc.item())
    print()

    time_elapsed=time.time()-since
    print('Validation complete in {:.0f}m
{:.0f}s'.format(time_elapsed//60, time_elapsed %60))
    print('Best Acc: {:.4f}'.format(best_acc))

    return acc_history

```

12. 예측 이미지 출력을 위한 전처리 함수

```

def im_convert(tensor):
    image=tensor.clone().detach().numpy()
    image=image.transpose(1,2,0)
    imge=image*(np.array((0.5,0.5,0.5))+np.array((0.5,0.5,0.5)))
    image=image.clip(0,1)

    return image

```

▼ 표 5-2 tensor.clone(), tensor.detach(), tensor.clone().detach()의 비교

구분	메모리	계산 그래프 상주 유무
tensor.clone()	새롭게 할당	계산 그래프에 계속 상주
tensor.detach()	공유해서 사용	계산 그래프에 상주하지 않음
tensor.clone().detach()	새롭게 할당	계산 그래프에 상주하지 않음

13. 개와 고양이 예측 결과 출력

```

classes = {0: 'cat', 1: 'dog'}

dataiter = iter(test_loader)
images, labels = dataiter.next()

output = model(images)
_, preds = torch.max(output, 1)

fig = plt.figure(figsize=(25, 4))

for idx in np.arange(20):
    ax = fig.add_subplot(2, 10, idx + 1, xticks=[], yticks=[])
    plt.imshow(im_convert(images[idx]))

    ax.set_title(
        "{}({})".format(
            str(classes[preds[idx].item()]),
            str(classes[labels[idx].item()])
        ),
        color=("green" if preds[idx] == labels[idx] else "red")
    )

plt.show()

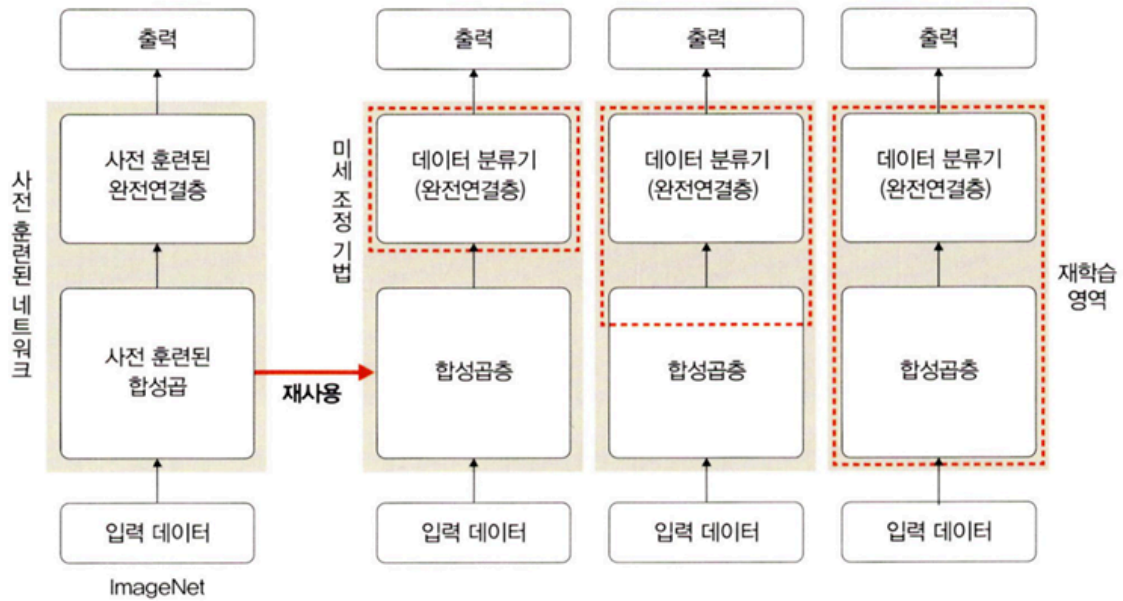
plt.subplots_adjust(bottom=0.2, top=0.6, hspace=0)

```

5.3.2 미세 조정 기법

미세 조정(fine-tuning)은 특성 추출 기법에서 더 나아가 사전 훈련된 모델과 합성곱층, 데이터 분류기의 가중치를 업데이트하여 훈련시키는 방법이다. 사전 학습된 모델을 목적에 맞게 재학습시키거나 학습된 가중치의 일부를 재학습시키는 것이다. 사전 훈련된 네트워크를 미세조정하여 분석하려는 데이터셋에 잘 맞도록 모델의 파라미터를 조정하는 기법이다.

▼ 그림 5-44 미세 조정 기법



- 데이터셋이 크고, 사전훈련된 모델과 유사성이 작을 경우: 모델 전체를 재학습, 데이터셋이 커서 재학습시키는 것이 좋은 전략
- 데이터셋이 크고 사전훈련된 모델과 유사성이 클 경우: 유사하기 때문에 합성곱층 뒷부분만 데이터 분류기만 새로 학습하더라도 좋은 전략
- 데이터셋이 작고 사전훈련된 모델과 유사성이 작을 경우: 데이터가 작아서 일부 계층에 미세조정기법을 적용해도 효과가 없을 수도 있다. → 합성곱층 일부분과 데이터 분류기를 학습, 합성곱층 중 어디까지 새로학습해야할지 적당히 설정해줘야한다.
- 데이터셋이 작고 사전훈련된 모델과 유사성이 클 경우 : 데이터 분류기만 학습 데이터셋이 작아서 과적합이 발생할 수도 있다. 최종 데이터 분류기인 완전연결층에 대해서만 미세조정기법 적용한다.

5.4 설명 가능한 CNN

설명 가능한 CNN(explainable CNN)은 딥러닝 처리 결과를 사람이 이해할 수 있는 방식으로 제시하는 기술이다. CNN 처리 과정을 시각화한다.

5.4.1 특성 맵 시각화

특성 맵 **feature map** 은 입력 이미지 필터를 입력에 적용한 결과이다.

1. 설명 가능한 네트워크 생성

```
class XAI(torch.nn.Module):
    def __init__(self, num_classes=2):
        super(XAI, self).__init__()

        self.features = nn.Sequential(
            nn.Conv2d(3, 64, kernel_size=3, bias=False),
            nn.BatchNorm2d(64),
            nn.ReLU(inplace=True),
            nn.Dropout(0.3),

            nn.Conv2d(64, 64, kernel_size=3, padding=1, bias=False),
            nn.BatchNorm2d(64),
            nn.ReLU(inplace=True),
            nn.MaxPool2d(kernel_size=2, stride=2),

            nn.Conv2d(64, 128, kernel_size=3, padding=1, bias=False),
            nn.BatchNorm2d(128),
            nn.ReLU(inplace=True),
            nn.Dropout(0.4),

            nn.Conv2d(128, 128, kernel_size=3, padding=1, bias=False),
            nn.BatchNorm2d(128),
            nn.ReLU(inplace=True),
            nn.MaxPool2d(kernel_size=2, stride=2),

            nn.Conv2d(128, 256, kernel_size=3, padding=1, bias=False),
            nn.BatchNorm2d(256),
            nn.ReLU(inplace=True),
            nn.Dropout(0.4),

            nn.Conv2d(256, 256, kernel_size=3, padding=1, bias=False),
            nn.BatchNorm2d(256),
            nn.ReLU(inplace=True),
            nn.Dropout(0.4),

            nn.Conv2d(256, 256, kernel_size=3, padding=1, bias=False),
            nn.BatchNorm2d(256),
            nn.ReLU(inplace=True),

            nn.MaxPool2d(kernel_size=2, stride=2),

            nn.Conv2d(256, 512, kernel_size=3, padding=1, bias=False),
            nn.BatchNorm2d(512),
            nn.ReLU(inplace=True),
```

```

        nn.Dropout(0.4),

        nn.Conv2d(512, 512, kernel_size=3, padding=1, bias=False),
        nn.BatchNorm2d(512),
        nn.ReLU(inplace=True),
        nn.Dropout(0.4),

        nn.Conv2d(512, 512, kernel_size=3, padding=1, bias=False),
        nn.BatchNorm2d(512),
        nn.ReLU(inplace=True),
        nn.MaxPool2d(kernel_size=2, stride=2),

        nn.Conv2d(512, 512, kernel_size=3, padding=1, bias=False),
        nn.BatchNorm2d(512),
        nn.ReLU(inplace=True),
        nn.Dropout(0.4),

        nn.Conv2d(512, 512, kernel_size=3, padding=1, bias=False),
        nn.BatchNorm2d(512),
        nn.ReLU(inplace=True),
        nn.Dropout(0.4),

        nn.Conv2d(512, 512, kernel_size=3, padding=1, bias=False),
        nn.BatchNorm2d(512),
        nn.ReLU(inplace=True),
        nn.MaxPool2d(kernel_size=2, stride=2),
    )

    self.classifier = nn.Sequential(
        nn.Linear(512, 512, bias=False),
        nn.Dropout(0.5),
        nn.BatchNorm1d(512),
        nn.ReLU(inplace=True),
        nn.Dropout(0.5),
        nn.Linear(512, num_classes)
    )

    def forward(self, x):
        x = self.features(x)
        x = x.view(-1, 512)
        x = self.classifier(x)

    return F.log_softmax(x)

```

- `log_softmax()` : `softmax` 함수 결과에 `log` 값을 취한 연산. 소프트맥스를 사용하지 않고 `log softmax`를 쓰는 이유는 `softmax`가 기울기 소멸 문제에 취약하기 때문이다.

3. 특성 맵을 확인하기 위한 클래스 정의

```
class LayerActivations:
    features=[]
    def __init__(self, model, layer_num):
        self.hook=model[layer_num].register_forward_hook(self.hook_fn)
    def hook_fn(self, module, input, output):
        self.features=output.detach().numpy()
    def remove(self):
        self.hook.remove()
```

- 파이토치에서는 매 계층마다 **print** 를 사용하지 않더라도 **hook** 기능을 통해 각 계층의 활성화 함수 및 기울기 값을 확인할 수 있다.

4. 이미지 호출

```
img=cv2.imread("/content/cat.jpg")
plt.imshow(img)
img=cv2.resize(img, (100,100), interpolation=cv2.INTER_LINEAR)
img=ToTensor()(img).unsqueeze(0)

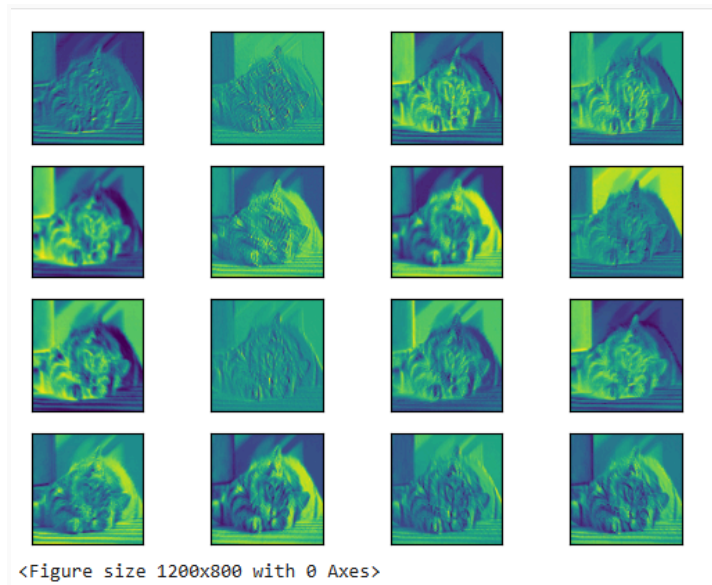
print(img.shape)
```

- **interpolation**:보간법, 이미지 크기를 변경할 경우 변형된 이미지의 픽셀을 추정해서 값을 할당해야하는데, 이때 새로운 픽셀값을 구하는 것을 보간법이라고 부른다.

5. 특성 맵 확인

```
fig, axes=plt.subplots(4,4)
fig=plt.figure(figsize=(12,8))
fig.subplots_adjust(left=0, right=1, bottom=0, top=1, hspace=0.05,
wspace=0.05)
for row in range(4):
    for column in range(4):
        axis=axes[row][column]
        axis.get_xaxis().set_ticks([])
        axis.get_yaxis().set_ticks([])
        axis.imshow(activations[0][row*10+column])
```

```
plt.show()
```



40번째 특성맵 사진은 다음과 같다. 뒤로 갈수록 원래 형태는 찾아보기 어렵고 이미지 특징들만 전달된다.



5.5 그래프 합성곱 네트워크

5.5.1 그래프란

그래프는 방향성이 있거나 없는 에지로 연결도니 노드의 집합이다.

5.5.2 그래프 신경망

그래프 신경망 **Graph Neural Netework, GNN** 은 그래프 구조에서 사용하는 신경망을 의미한다. 인접행렬과 특성행렬의 두단계로 이루어진다.

1 단계: 인접 행렬 **adjacency matrix**

노드 n 개를 $n \times n$ 행렬로 표현한다. 인접 행렬 내의 값은 $A_{ij}=i$ 와 j 의 관련성 여부를 만족하는 값으로 채운다.

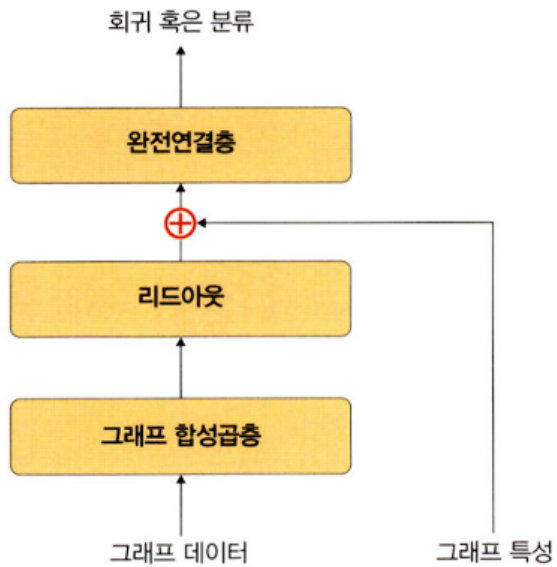
2 단계: 특성 행렬 **feature matrix**

인접 행렬만으로는 특성을 파악하기 어려워 단위 행렬을 적용한다. 각 입력 데이터에서 이용할 특성을 선택한다. 특성 행렬에서 각 행은 선택된 특성에 대해 각 노드가 갖는 값을 의미한다.



5.5.3 그래프 합성곱 네트워크

그래프 합성곱 네트워크 **GCN** 은 이미지에 대한 합성곱을 그래프 데이터로 확장한 알고리즘이다.



리드아웃(readout)은 특성행렬을 하나의 벡터로 변환하는 함수이다.

GCN에서 가장 중요한 부분은 그래프 합성곱층이다. 그래프 합성곱층을 이용한 그래프 형태의 데이터는 행렬 형태의 데이터로 변환되어 딥러닝 알고리즘을 적용할 수 있기 때문이다. 보통 SNS에서의 관계 네트워크, 학술 연구에서의 인용 네트워크 같은 곳에서 활용된다.